

Efficient Layer-Granularity Unloading for LLMs in Edge Computing

Zhenzheng Li, Zhiqing Tang, *Member, IEEE*, Jianxiong Guo, *Member, IEEE*,
Weijia Jia, *Fellow, IEEE*, and Wei Zhao, *Fellow, IEEE*

Abstract—Advancements in edge computing and container technology have made it increasingly popular and convenient to deploy Large Language Models (LLMs) through containers at the edge. However, the limited GPU resources of edge servers make it impractical to retain the model in GPU memory for long periods due to the high memory cost, especially when they remain idle without user requests. Existing work unloads the entire idle models to reduce memory costs on edge servers, but reloading them introduces significant loading delays that affect task Quality of Service (QoS). Therefore, efficient management of idle models is a critical issue that has been largely neglected in existing research and requires urgent attention. To address this gap, this paper studies the problem of idle model management from the perspective of the trade-off between memory cost and loading delay under the QoS constraint. A novel layer-granularity model unloading method is proposed, which leverages the layered characteristics of the model. We formulate an online joint optimization problem to determine which layers to unload and when, and present a layer-granularity unloading strategy inspired by the ski rental problem to solve it. We implement a real system with layer-granularity unloading for LLMs on NVIDIA GPUs and validate the effectiveness of the proposed method. Experimental results show it effectively trades off memory cost and loading delay, improving overall performance by up to 39.6%.

Index Terms—LLM, model unloading, layer-granularity, edge computing, online joint optimization

I. INTRODUCTION

The emergence of Large Language Models (LLMs) has enabled various applications, such as real-time question answering systems and intelligent personal assistants [1], [2]. To address the needs for fast responses and data privacy for these applications, deploying LLMs at the edge has become essential [3]. Using container technology [4], [5], it can efficiently deploy models at the edge. This process involves starting a container with the appropriate environment (e.g., PyTorch

[6], Transformer [7]) and mounting the parameter files of the model to the storage volume [8]. The model is then loaded into GPU memory¹ for autoregressive inference. In this scenario, the overall first-token response delay primarily consists of two components: the model loading delay and the prefill delay. For example, on an edge server equipped with PCIe 3.0, loading a 7B-parameter LLM from disk to memory takes approximately 4800 ms, whereas the prefill delay for 512 tokens is less than 50 ms. Therefore, the loading delay dominates the overall first-token response delay, enforcing a bound on loading delay effectively enforces the QoS requirement on the service delay. To ensure QoS, this paper defines the QoS constraint as maintaining the model loading delay within a specified threshold. If a user request arrives when the model is not resident in memory, loading a model with billions of parameters from scratch can lead to significant QoS violations. To this end, the persist retention method maintains the model by deploying it as a long-running container [8], thereby keeping it permanently. This avoids the need for reloading and guarantees that the QoS constraint is satisfied.

Nonetheless, due to the limited memory resources of the edge servers [9], it becomes impractical to retain the model in memory for extended periods, especially when the model has not been handling tasks for a while. Specifically, LLM services often experience highly variable and sparse request arrivals [10]. To accommodate peak demand, service providers commonly overprovision multiple LLM instances. However, empirical observations show that these instances frequently remain idle for extended off-peak periods, during which retaining full models in memory incurs substantial memory costs. Saving memory resources is highly attractive as it allows edge servers to utilize free memory for low-priority tasks [11], thereby increasing overall resource utilization. Therefore, finding a balance between model loading delay and memory resource usage while satisfying the QoS constraint is an urgent issue that needs to be addressed.

Several existing container management methods, such as fixed retention and histogram policies [12]–[14], to kill the container (i.e., unload the model) after a set time to reduce memory costs. However, the requests made after the model has been unloaded will result in QoS violations due to the need to reload the model, making model retention decisions a pervasive performance bottleneck in edge LLM serving.

¹In this paper, both “GPU memory” and “memory” refer to the GPU’s device memory.

Zhenzheng Li is with School of Artificial Intelligence, Beijing Normal University, Beijing 100875, China, and also with Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China. (E-mail: zhenzhengli@mail.bnu.edu.cn)

Zhiqing Tang is with Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China. (E-mail: zhiqing-tang@bnu.edu.cn)

Jianxiong Guo and Weijia Jia are with Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China, and also with Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, Zhuhai 519087, China. (E-mail: {jianxiongguo, jiawj}@bnu.edu.cn)

Wei Zhao is with Shenzhen University of Advanced Technology, Shenzhen 518055, China. (E-mail: zhao.wei@siat.ac.cn) (*Corresponding authors: Zhiqing Tang and Weijia Jia.*)

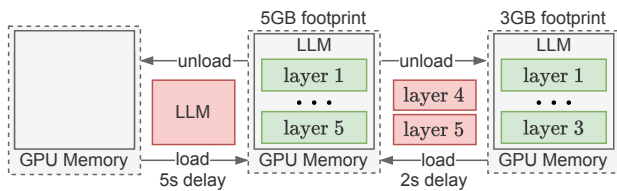


Fig. 1. Model-granularity unloading versus the layer-granularity unloading.

In brief, existing approaches address the problem from the perspective of the entire model, either retaining the whole model in memory or unloading it entirely, resulting in high memory costs or QoS violations. *In fact, these approaches seriously overlook the layered structure of the models [15], [16], which offers the potential for layer-granularity unloading.* Fig. 1 illustrates an example of the difference between layer-granularity unloading and model-granularity unloading. By retaining only layers 1-3, memory footprints are reduced compared to retaining the entire model. When a new request arrives, only the unloaded layers 4-5 need to be reloaded, which reduces the loading delay compared to reloading the entire model, thereby satisfying the QoS constraint. Therefore, leveraging the layered structure of LLM allows us to unload the model with finer granularity, addressing the dilemma between QoS violations and high memory costs.

Retaining or unloading the LLM at the layer granularity shows great promise, but several challenges must be addressed. *The first challenge is how to unload or load the models at the layer granularity.* To address this, we utilize the AirLLM [17] code to implement this mechanism that creates a virtual *meta* device [18] for managing the loading and unloading of model layers between the disk and memory. Specifically, the *meta* device is used to create placeholder tensors that do not occupy actual memory; these tensors retain only structure information such as shape and data type, without allocating memory for their contents. Layer loading is implemented by transferring the layer's parameters from the *meta* device to memory for efficient disk-to-memory loading, while layer unloading is implemented by setting the layer back to the *meta* device, thereby freeing up its memory footprint.

The second challenge is how to determine which layers to unload and when, to trade off the loading delay and memory cost. Specifically, our goal is to jointly minimize the overall cost (i.e., the weighted model loading delay and the memory cost during idle periods), while ensuring that the loading delay to serve any request does not exceed a predefined QoS threshold. Retaining the layers in memory can reduce loading delays if requests are expected soon, but it also incurs significant memory costs if the requests arrive much later [19]. The memory cost can be regarded as a resource expenditure during the model's idle periods in exchange for the benefit of reduced loading delay. Consequently, deciding the unloading time for layers involves a trade-off [19]–[21] between loading delay and memory cost. However, the decision process is extraordinarily complex due to the irregular and intermittent nature of edge requests and the uncertainty of future request arrivals [22]. To this end, we formulate an

online joint optimization problem to jointly minimize loading delay and memory cost by determining which layers to unload and when, under the QoS constraint. To solve this problem, a Layer-granularity Unloading Algorithm (LUA) is proposed. We transform the joint optimization problem for multiple layers into a multi-slope ski rental problem [23] and reduce the complexity to a 2-slope ski rental problem. LUA leverages statistics on recent request arrival intervals, specifically the long-interval probability, as well as the first-order and second-order moments of short intervals, to derive a strategy for the 2-slope ski rental problem using the Lagrange duality method. Based on this strategy, the LUA selects the strategy with the minimal objective cost to determine the unloading time for the layers.

In this paper, we develop a layer-granularity unloading system, which comprises the Loading Engine, Unloading Scheduler, and Unloading Engine as its primary components. Layer granularity retention and unloading are implemented by the *meta* device feature provided by HuggingFace Accelerate [18]. The Unloading Scheduler, a core system component, determines which layers to retain and when to unload layers using the LUA. We have implemented a testbed with a NVIDIA GeForce RTX 3090 GPU with 24 GB of memory, a RTX 2070 SUPER GPU with 8 GB of memory, and a NVIDIA Jetson AGX Orin with 64 GB of memory to validate the effectiveness of LUA. Experimental results demonstrate that LUA effectively trades off memory cost and loading delay, reducing overall costs by up to 39.6% compared with the baselines. Unlike inference acceleration works, this paper focuses on idle-time model residency management, a previously underexplored bottleneck in edge LLM serving. The contributions of this paper are as follows:

- We present a layer-granularity unloading mechanism for LLMs in edge computing. An online optimization problem is formulated to jointly minimize loading delay and memory cost while adhering to the QoS constraint.
- We propose the LUA, simplifying the joint optimization problem to a 2-slope ski rental problem and incorporating statistics to develop the layer unloading strategy.
- We implement a real unloading system. The experimental results demonstrate that our proposed LUA reduces overall costs by up to 39.6% compared to the baselines.

II. MOTIVATION AND RELATED WORK

A. Motivation Example

Compared to coarse model granularity, making retention and unloading decisions at the layer granularity offers more substantial advantages. To illustrate this, we provide a motivation example. Consider a model with five layers, each with a loading delay and memory footprint of 1 s and 1 GB, respectively. The QoS constraint is defined as a loading delay of no more than 2 s. After the service of the user request is completed, the model enters an idle state, and the model granularity unloading scheme needs to choose between unloading the entire model or retaining it at this point. If the choice is to retain the entire model, then the 5 GB memory footprint will incur a significant memory cost during the idle

time. Conversely, if the model is unloaded, user requests arriving after the model has been unloaded require the entire model to be reloaded, which incurs a loading delay of 5 s, thereby violating the QoS constraint. In contrast, if the layer granularity unloading scheme is enabled, unloading two layers reduces the memory footprint by 2 GB. Only 2 s are needed to reload these two layers when a new request arrives, thereby adhering to the QoS constraint.

TABLE I
THE OVERALL COST FOR DIFFERENT LAYER UNLOADING STRATEGIES

Requests	Decisions	Delay	Memory cost	Overall cost
20s	$t = 10, t' = 10$	2s	80	140
	$t = 10, t' = 30$	1s	90	120
	$t = 30, t' = 30$	0s	100	100
35s	$t = 10, t' = 10$	2s	125	185
	$t = 10, t' = 30$	2s	145	205
	$t = 30, t' = 30$	2s	165	225

Moreover, the unloading time of the layers impacts both memory cost and loading delay when serving the next request. We define the overall cost as the weighted sum of loading delay and memory cost, with a weight of 30. TABLE I illustrates the overall cost for different decisions when the requests arrive in 20 seconds and 35 seconds, respectively, where t and t' denote the unloading times of these two layers. The results show that the overall cost is minimized when the unloading time is set to $t = 30, t' = 30$ and the next request arrives at 20 seconds, but this strategy yields the worst cost when the arrival time occurs at 35 seconds. Conversely, setting the unloading time to $t = 10, t' = 10$ minimizes the cost when the next request arrives at 35 seconds, but results in the highest cost when the arrival time occurs at 20 seconds. The difficulty lies in the unpredictability of future request arrivals, complicating the decision-making process. Therefore, the decision on layer unloading time must be carefully made to trade-off loading delay and memory cost.

B. Related Work

LLM Retention Management. Several studies investigate the retention management problem of LLMs. Xu *et al.* [24] present a joint caching and inference framework for edge intelligence that enhances LLM agent provisioning using a cached model-as-a-resource approach. TrimCaching [25] proposes a parameter-sharing model caching scheme for edge servers in mobile networks, maximizing cache hit ratios and enhancing storage efficiency through shared parameter blocks among models. S-LoRA [26] improves LLM deployment by caching adapters in main memory and dynamically managing them in memory through unified paging, tensor parallelism, and optimized CUDA kernels. However, these methods mostly operate at the granularity of entire models or adapters. They assume that either full models or adapters are cached or evicted.

LLMs in Edge Computing. Deploying LLMs at the edge is becoming increasingly popular, with several studies investigating this topic. HALO [27] improves the execution efficiency of LLM at the edge by utilizing a heterogeneous 2.5D

chiplet-based architecture, primarily targeting inference-time acceleration and energy efficiency through specialized hardware and system-level optimizations. EdgeShard [28] utilizes collaborative edge computing to partition LLM, enhancing inference efficiency with a dynamic programming algorithm. VELO [29] uses vector database caching and a multi-agent reinforcement learning algorithm to optimize QoS, reducing response times and costs for LLM requests at the edge. T-MAC [30] presents a lookup table approach for efficient low-bit LLM inference on CPUs, enabling mixed precision matrix multiplication without dequantization. PerLLM [31] utilizes edge-cloud collaboration to optimize LLM service scheduling and resource allocation, achieving higher throughput and reducing energy costs. He *et al.* [32] propose an active inference-based approach for the offloading and resource allocation of LLM inference tasks in cloud-edge computing, addressing the inefficiency problem. While these studies explore sharding, scheduling, and quantization-based efficiency, they assume that model components are static once deployed. In contrast, this paper focuses on the problem of idle-time resource management, which is complementary to such techniques and can be integrated with them to further improve overall system efficiency in edge environments. Our work studies the problem of idle model management from the perspective of the trade-off between memory cost and loading delay under the QoS constraint, explores layer-granularity unloading, and enables finer control, which is especially beneficial in highly constrained edge environments. By doing so, our proposed solutions effectively address these gaps.

III. SYSTEM OVERVIEW

The layer-granularity unloading system framework is depicted in Fig. 2, where the primary components include the Endpoint, Inference Engine, Loading Engine, Unloading Scheduler, and Unloading Engine. The Unloading Scheduler is the core system component to specify which layers are retained and when layers are unloaded.

Layer-granularity Implementation. Several critical issues need to be addressed in order to implement layer-granularity retention and unloading. First, how to divide the model's parameter files into layers. Models typically consist of a fixed number of layers, while the raw model's parameter files are generally stored in multiple chunks rather than by individual layers, with each chunk containing multiple layers. Loading layers via chunks may cause redundant loading of some layers, resulting in significant disk read delays and I/O inefficiencies. To address this, we preprocess the original model parameter files by reorganizing them such that each storage block contains parameters of exactly one layer, stored in the `safetensor` format [33]. This allows the model to be loaded layer-by-layer by loading the corresponding storage block. Notably, this preprocessing step does not modify the model architecture or inference logic and is fully compatible with existing HuggingFace Transformer models. Therefore, this mechanism can be applied to general LLMs without changing their network structure.

Second, how to load or unload layers between the disk and memory. We reference the AirLLM code [17] to implement a

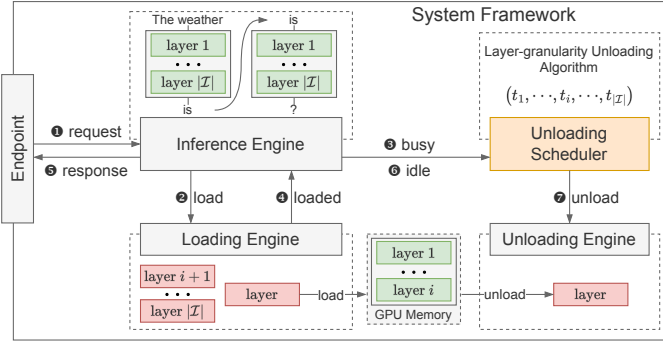


Fig. 2. System framework: ❶ The Endpoint transmits the request to the Inference Engine. ❷ The Inference Engine instructs the Loading Engine to load the required layers. Concurrently, ❸ a busy event is sent to the Unloading Scheduler to prevent the layers from being unloaded. Once loading is complete, ❹ it informs the Inference Engine, commencing autoregressive inference. ❺ The Inference Engine returns the response to the Endpoint. After inference completion, ❻ the Inference Engine issues an idle event to the Unloading Scheduler. ❼ Based on the LUA, the Unloading Scheduler sends the unload events to the Unloading Engine to unload layers.

layered loading and unloading mechanism. The `meta` device is a virtual device provided by HuggingFace Accelerate [18] that allows tensors to be instantiated without allocating physical memory. These tensors preserve only structure information such as shape and data type. When a layer is transferred from the `meta` device to memory, its parameters are loaded from disk and materialized in memory. Conversely, moving a layer back to the `meta` device releases the occupied memory while preserving the model structure. In this way, the `meta` device serves as a lightweight bridge between storage and memory, enabling fine-grained loading and unloading of model layers. The remainder of this section provides a detailed illustration of each system component.

Endpoint. The Endpoint transmits requests to the Inference Engine and processes the responses. It serves as the interface between the system and external entities, forwarding request tokens to the Inference Engine and awaiting responses. Once received, the Endpoint sends the responses back to the external requester.

Inference Engine. It processes requests and performs autoregressive inference. Upon receiving a request, the Inference Engine checks whether all necessary layers are in memory. If they are, inference starts immediately. If not, it sends a load event to the Loading Engine to load the missing layers and a busy event to the Unloading Scheduler to stop any unloading processes. Autoregressive inference begins only after receiving a loaded event from the Loading Engine. After completing the inference, it sends an idle event to the Unloading Scheduler. Additionally, the Inference Engine tracks request history to gather statistics on request intervals, which the LUA uses to optimize unloading decisions.

Loading Engine. It operates as a daemon tasked with loading the necessary layers of the model. Upon receiving a load event, the Loading Engine identifies and loads any layers that are not currently retained in memory. By transferring the layer from the `meta` device to memory, the layer is actually loaded into memory. Once the loading is complete, it sends a

TABLE II
MAIN NOTATIONS

Notations	Definition
$\mathcal{I}$	Set of layers of the model
$(t_1, \dots, t_i, \dots, t_{ \mathcal{I} })$	The unloading time of layers
c_i	Memory footprint of layer i
d_i	Loading delay of layer i
s_i	Retention state of the model
C_i	Memory footprint of the model in state s_i
D_i	Loading delay of the model in state s_i
D_{QoS}	QoS constraint
η	Weight of the loading delay
q_{B+}	Long-interval probability
μ_{B-}	Short-interval first-order moment
σ_{B-}	Short-interval second-order moment

loaded event to the Inference Engine to confirm that the layers have been successfully loaded.

Unloading Scheduler. It determines which layers are to be retained and specifies when to unload the layers during periods of model idleness. Upon receiving an idle event, the Unloading Scheduler calculates the unloading time for each layer using the LUA, which will be detailed in Section IV. Consider an LLM comprising $|\mathcal{I}|$ layers, where $\mathcal{I}$ is the set of layers. The unloading times for these layers are denoted by $(t_1, \dots, t_i, \dots, t_{|\mathcal{I}|})$, where $i \in \mathcal{I}$. Note that if $t_i = +\infty$, it indicates that the layer i is always retained in memory. Assume the model becomes idle at $t = 0$ and denote the current time as t . During the model's idle period, if $t \geq t_i$, an unload event is triggered and sent to the Unloading Engine to unload layer i . The Unloading Scheduler terminates any ongoing unloading processes if a busy event is received.

Unloading Engine. It is a daemon that executes the unloading of the layers. By setting the device variable of the layer from memory to the `meta` device, the layer is unloaded, and its memory footprint is released. In principle, the Loading Engine and Unloading Engine can be unified into a single Memory Manager module; however, we separate them in our implementation to improve modularity and simplify system design. Additionally, the Unloading Engine maintains a status table that records information about the layers. The status table is updated immediately after a layer is successfully unloaded or reloaded. Specifically, upon completing a layer unloading operation, the Unloading Engine marks the corresponding layer as unavailable in memory; similarly, after a loading operation is completed, the layer status is updated to indicate availability. The Inference Engine consults this table before inference to ensure that all required layers are present in memory.

IV. ONLINE ALGORITHM DESIGN

As described in Section III, the Unloading Scheduler is a core component, and the LUA within it is responsible for determining the layers' unloading times $(t_1, \dots, t_i, \dots, t_{|\mathcal{I}|})$. In this section, we first formulate an online joint optimization problem, subsequently transforming it into a ski rental problem. Finally, the LUA is proposed to solve the problem. The primary notations utilized in this paper are shown in TABLE II.

A. System Model

Consider an LLM with $|\mathcal{I}|$ layers. If layer i of the model is retained in memory, it occupies a memory footprint of c_i . Consequently, retaining the entire model requires memory equivalent to $\sum_{i \in \mathcal{I}} c_i$. When a new request arrives, all parameters required for the current inference must be resident in memory. If layer i is missing (i.e., unloaded), it must be reloaded, incurring a loading delay denoted as d_i . Thus, loading the entire model results in a delay of $\sum_{i \in \mathcal{I}} d_i$. After completing the autoregressive inference, the model becomes idle, i.e., with no ongoing inference. The memory cost is defined as the product of the memory footprint during the idle and the duration of occupancy. Layers should be unloaded as early as possible to reduce the memory cost. Conversely, layers must be retained in memory to reduce loading delays for subsequent requests. Therefore, the memory cost can be regarded as a resource expenditure during the model's idle periods in exchange for the benefit of reduced loading delay. This trade-off requires carefully determining $(t_1, \dots, t_i, \dots, t_{|\mathcal{I}|})$ to jointly optimize loading delay and memory cost.

To facilitate analysis, we define the retention state s_i of the model based on the retention status of its layers in memory. The model is in state s_i when layers 1 to i are retained and layers $i+1$ to $|\mathcal{I}|$ are unloaded. All layers are retained when the model is in state $s_{|\mathcal{I}|}$; in state s_0 , all layers are unloaded. Transitioning from state s_i to state s_{i-1} implies that layer i is unloaded. Thus, determining t_i corresponds to determining the time to transition from state s_i to state s_{i-1} . In state s_i , the model occupies memory of $C_i = \sum_{i' \leq i} c_{i'}$. When a request arrives, the model must reload the necessary layers to return to state $s_{|\mathcal{I}|}$. Thus, serving a request from state s_i incurs a loading delay of $D_i = \sum_{i' > i} d_{i'}$. The state with a larger memory footprint has a smaller loading delay, and vice versa. Therefore, it holds that $C_0 < \dots < C_{|\mathcal{I}|}$ and $D_0 > \dots > D_{|\mathcal{I}|}$.

B. QoS Constraint

The QoS constraint specifies that the loading delay must not exceed D_{QoS} . Consequently, any state s_i where $D_i > D_{QoS}$ will be prohibited due to excessive loading delays, violating the QoS constraint. We identify the minimal i that satisfies $D_i \leq D_{QoS}$, denoted as $\bar{i} = \{\min_{i \in \mathcal{I}} \{i\} | D_i \leq D_{QoS}\}$. Being in states s_0 to $s_{\bar{i}-1}$ would violate the QoS constraint, meaning that, to satisfy QoS requirements, layers 1 to $\bar{i}$ must always be retained in memory. Consequently, we have:

$$\{t_i = +\infty | 1 \leq i \leq \bar{i}, \bar{i} = \{\min_{i \in \mathcal{I}} \{i\} | D_i \leq D_{QoS}\}\}. \quad (1a)$$

C. Problem Formulation

The remaining problem involves determining the unloading time for layers $\bar{i}+1$ to $|\mathcal{I}|$. Assume the model becomes idle at $t = 0$. Let $S_i(t)$ denote the probability that the model is in state s_i at t , where $\sum_{i \in \mathcal{I} \cup \{0\}} S_i(t) = 1$ for $t \geq 0$. To satisfy the QoS constraint, it is required that $S_i(t) = 0$ for

$0 \leq i \leq \bar{i} - 1$. For any next request arriving at τ , the online joint optimization problem is given by:

$$\min \sum_{i \in \mathcal{I} \cup \{0\}} \eta D_i S_i(\tau) + \int_{0^+}^{\tau} \sum_{i \in \mathcal{I} \cup \{0\}} C_i S_i(t) dt, \quad (2a)$$

$$\text{s.t.} \sum_{i \in \mathcal{I} \cup \{0\}} S_i(t) = 1, \quad t \geq 0, \quad (2b)$$

$$S_i(t) = 0, \quad 0 \leq i \leq \bar{i} - 1, \quad (2c)$$

where η is the weight assigned to the loading delay. The optimization objective is to minimize the weighted sum of (1) the loading delay incurred when serving the next request and (2) the memory cost accumulated during the model's idle period, subject to the QoS constraint on the maximum allowable loading delay. The weighted objective reflects the inherent trade-off between memory cost during idle periods and loading delay upon request arrival. The weight parameter η converts loading delay into an equivalent memory cost scale. This weighted objective unifies heterogeneous costs via weighting, a common formulation in resource management and scheduling systems [20]. By minimizing the overall cost to jointly optimize loading delay and memory cost, we can achieve the trade-off between these two factors. The main challenge in solving the problem is the uncertainty of the future request arrival time τ , which is unknown when the decision is made. This complicates the determination of the unloading decision. Next, we demonstrate that this joint optimization problem can be transformed into the multi-slope ski rental problem [23].

D. Transform to the Ski Rental Problem

In the multi-slope ski rental problem, a skier with an uncertain skiing duration must select from various options to minimize costs. Each ski rental option is characterized by an initial buying price (analogous to the loading delay for a retention state) and an ongoing rental cost (analogous to the memory footprint for a retention state). Being in a certain state of the model corresponds to selecting a ski rental option. The unpredictable arrival time of the next request is analogous to the uncertain duration of skiing. Therefore, the layer unloading problem can be transformed into a multi-slope ski rental problem.

Furthermore, we demonstrate that this problem can be simplified to a 2-slope ski rental problem. In the 2-slope ski rental problem, a skier must either rent a ski for the day or buy one. In LLMs, the loading delay of a layer is primarily determined by its parameter size and the effective bandwidth of the disk-to-GPU data path (e.g., PCIe). As a result, the loading delay of the layer is approximately proportional to its memory footprint, i.e., $d_i = kc_i$. Although minor variations may arise due to fluctuations, this proportional relationship holds in practice, as validated by our empirical measurements in TABLE III. If the model consistently remains in state s_{i_1} and s_{i_2} , the overall cost incurred until time t is $\sum_{i' \leq i_1} c_{i'} t + \sum_{i' > i_1} \eta d_{i'}$

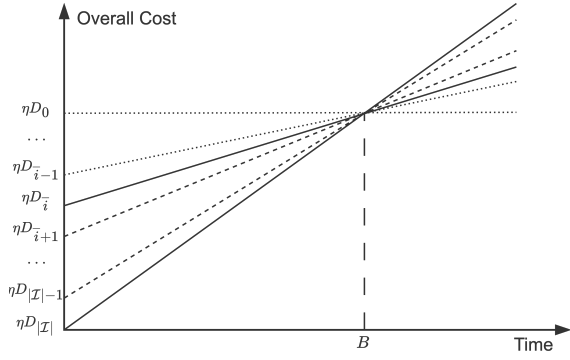


Fig. 3. An illustration of a multi-slope ski rental problem is presented, with each line representing different states of the model. (1) The slope of each line indicates the memory footprint of the model in a given state, while the intercept represents the weighted loading delay for that state. (2) The dotted lines correspond to states that do not meet the QoS constraint, whereas the other lines (dashed and solid lines) represent states that satisfy the QoS constraint. (3) For $s_{\bar{i}} = \arg \min_{s_i} \{C_i \tau + \eta D_i\}$ if $\tau > B$ and $s_{|\mathcal{I}|} = \arg \min_{s_i} \{C_i \tau + \eta D_i\}$ if $\tau \leq B$, only states $s_{\bar{i}}$ and $s_{|\mathcal{I}|}$ will be chosen. These two states are mapped to the buy and rent options of the 2-slope ski rental problem, respectively, where renting costs 1 and buying costs B .

and $\sum_{i' \leq i_2} c_{i'} t + \sum_{i' > i_2} \eta d_{i'}$, respectively. The intersection of these two lines occurs at:

$$\sum_{i' \leq i_1} c_{i'} t + \sum_{i' > i_1} \eta d_{i'} = \sum_{i' \leq i_2} c_{i'} t + \sum_{i' > i_2} \eta d_{i'}. \quad (3a)$$

By substituting $d_i = k c_i$, we obtain $t = \eta k$. This indicates that regardless of which pair of states s_{i_1} and s_{i_2} , they intersect at the same point B , as illustrated in Fig. 3. Here, $B = \eta D_{\bar{i}} / (C_{|\mathcal{I}|} - C_{\bar{i}})$ is the intersection of these lines. Since $s_{\bar{i}} = \arg \min_{s_i} \{C_i \tau + \eta D_i\}$ if $\tau > B$ and $s_{|\mathcal{I}|} = \arg \min_{s_i} \{C_i \tau + \eta D_i\}$ if $\tau \leq B$, it is optimal to be in state $s_{\bar{i}}$ if the next request arrives at $\tau > B$. Conversely, it is optimal to be in state $s_{|\mathcal{I}|}$ if the next request arrives at $\tau \leq B$. No other state will be selected, indicating that layers $\bar{i} + 1$ to $|\mathcal{I}|$ must be unloaded simultaneously at time $\bar{t}$, i.e., $\bar{t} = t_{\bar{i}+1} = \dots = t_{|\mathcal{I}|}$. Therefore, we map the states $s_{\bar{i}}$ and $s_{|\mathcal{I}|}$ to the buy and rent options of the 2-slope ski rental problem, respectively. The renting cost is 1, while the buying cost is B .

E. Strategy for the Ski Rental Problem

To determine $\bar{t}$, this subsection presents a strategy to solve the 2-slope ski rental problem. We define a probability density function $P(t)$ that denotes the probability of unloading layers $\bar{i} + 1$ to $|\mathcal{I}|$ at time t . As analyzed previously, the model must transition from state $s_{|\mathcal{I}|}$ to state $s_{\bar{i}}$ within the interval $0 \leq t \leq B$, so $P(t)$ is defined only for $0 \leq t \leq B$ and is 0 for all other cases. The probability density function $P(t)$ is given as:

$$P(t) = p(t) + \alpha \delta(t) + \beta \delta(t - B), \quad (4a)$$

where $p(t)$ is the probability density function for $0 < t < B$. There are two discrete distribution components at $t = 0$ and $t = B$ with probability masses α and β , respectively. The Dirac delta function is used to represent the discrete probability distributions. Several studies have demonstrated that

additional statistics can enhance algorithm performance [34]–[36]. Therefore, we incorporate statistics on request arrival intervals, including the long-interval probability q_{B+} , short-interval first-order moment μ_{B-} , and short-interval second-order moment σ_{B-} . Although variance is typically easier to obtain, we use second-order moments here because variance and the second-order moments differ only by a constant. Unlike [34]–[36], which uses one of q_{B+} , μ_{B-} , and σ_{B-} as the additional input to derive the strategy, our goal is to extend these approaches. We aim to incorporate q_{B+} , μ_{B-} , and σ_{B-} together to derive a concrete expression for $P(t)$, i.e., to determine $p(t)$, α , and β . The statistics are as follows:

$$\int_{0+}^B q(\tau) d\tau + q_{B+} = 1, \quad (5a)$$

$$\int_{0+}^B \tau q(\tau) d\tau = \mu_{B-}, \quad (5b)$$

$$\int_{0+}^B \tau^2 q(\tau) d\tau = \sigma_{B-}, \quad (5c)$$

where τ and $q(\tau)$ are the arrival time and the arrival time's probability density function of the next request, respectively. The expected cost for $\tau \leq B$ obtained from $P(t)$ is given by:

$$C(P(t), \tau \leq B) = \int_{0+}^{\tau} (t + B) P(t) dt + \int_{\tau}^B \tau P(t) dt, \quad (6a)$$

and the expected cost for $\tau > B$ obtained from $P(t)$ is:

$$C(P(t), \tau > B) = \int_{0+}^B (t + B) P(t) dt. \quad (7a)$$

Therefore, using (6a) and (7a), the expected online cost can be written as follows:

$$J(P(t), q(\tau)) = \int_{0+}^B C(P(t), \tau \leq B) q(\tau) d\tau + C(P(t), \tau > B) q_{B+}. \quad (8a)$$

The adversary will choose a $q(\tau)$ to maximize the expected online cost. Conversely, our goal is to derive $P(t)$ such that the worst-case expected online cost $J(P(t), q(\tau))$ is minimized. Consequently, it can be formulated as a minimax problem:

$$\min_{P(t)} \max_{q(\tau)} J(P(t), q(\tau)), \quad (9a)$$

$$\text{s.t.} \quad (5a), (5b), (5c),$$

$$\int_{0+}^B p(t) dt = 1 - \alpha - \beta, \quad (9b)$$

where the constraint (9b) ensures that $P(t)$ is a valid probability density function. To solve the minimax problem, we construct the dual of the maximization problem. By associating the constraints on $q(\tau)$ with Lagrange multipliers and substituting $P(t)$ with (4a), we derive the Lagrangian for the maximization problem:

$$\begin{aligned} L(q(\tau), \lambda_1, \lambda_2, \lambda_3) &= q_{B+} \int_{0+}^B (t + B) p(t) dt + \alpha q_{B+} B \\ &+ \int_{0+}^B [C(P(t), \tau \leq B) - \lambda_1 - \lambda_2 \tau - \lambda_3 \tau^2] q(\tau) d\tau \\ &+ 2\beta q_{B+} B + \lambda_1 (1 - q_{B+}) + \lambda_2 \mu_{B-} + \lambda_3 \sigma_{B-}, \end{aligned} \quad (10a)$$

where the Lagrange multipliers λ_1 , λ_2 , and λ_3 are associated with the constraints (5a), (5b), and (5c), respectively. The dual function $g(\lambda_1, \lambda_2, \lambda_3) = \sup_{q(\tau)} L(q(\tau), \lambda_1, \lambda_2, \lambda_3)$ has an upper bound only if $C(P(t), \tau \leq B) - \lambda_1 - \lambda_2\tau - \lambda_3\tau^2 = 0$ holds. Consequently, the resulting dual problem is:

$$\min_{\lambda_1, \lambda_2, \lambda_3} \left[q_{B+} \int_{0+}^B (t+B)p(t)dt + \alpha q_{B+} B \right. \\ \left. + 2\beta q_{B+} B + \lambda_1(1 - q_{B+}) + \lambda_2\mu_{B-} + \lambda_3\sigma_{B-} \right], \quad (11a)$$

$$\text{s.t. } C(P(t), \tau \leq B) - \lambda_1 - \lambda_2\tau - \lambda_3\tau^2 = 0, \quad (11b)$$

$$0 \leq \lambda_1, 0 \leq \lambda_2, 0 \leq \lambda_3. \quad (11c)$$

Next, we transform λ_1 , λ_2 , and λ_3 into functions of α and β by solving the dual problem, subsequently converting it into an optimization problem in terms of α and β . To derive $p(t)$, we differentiate (11b) twice with respect to τ and substitute τ with t , resulting in the following ordinary differential equation:

$$\frac{dp(t)}{dt} = \frac{1}{B}(p(t) + 2\lambda_3). \quad (12a)$$

Considering the constraint (9b) as the boundary condition, the solution to this ordinary differential equation is given as:

$$p(t) = c_0 e^{t/B} - 2\lambda_3, \quad (13a)$$

$$c_0 = \frac{1 - \alpha - \beta + 2\lambda_3 B}{B(e-1)}. \quad (13b)$$

Furthermore, to maintain a positive probability density function, the following constraints must be imposed on λ_3 :

$$\lambda_3 \leq \frac{1 - \alpha - \beta}{2B(e-2)}. \quad (14a)$$

By substituting (13a) into (11b), the resulting equations are:

$$\lambda_1 = \alpha B, \quad (15a)$$

$$\lambda_2 = \beta + \frac{e}{e-1}(1 - \alpha - \beta) + \frac{(4-2e)B}{e-1}\lambda_3, \quad (15b)$$

where λ_1 is expressed as a function of α . To derive λ_2 and λ_3 as a function of α and β , we substitute constraint (11b) with (15b), while incorporating (14a) as an additional constraint. Furthermore, we substitute (13a) into (11a). This leads to an equivalent formulation of the dual problem:

$$\min_{\lambda_2, \lambda_3} \frac{(1 - \alpha - \beta)e q_{B+} B}{e-1} + \alpha B + 2\beta q_{B+} B \\ + \lambda_2 \mu_{B-} + \lambda_3 \left(\frac{(3-e)q_{B+} B^2}{e-1} + \sigma_{B-} \right), \quad (16a)$$

$$\text{s.t. } \lambda_2 = \beta + \frac{e}{e-1}(1 - \alpha - \beta) + \frac{(4-2e)B}{e-1}\lambda_3, \quad (16b)$$

$$0 \leq \lambda_2, 0 \leq \lambda_3 \leq \frac{1 - \alpha - \beta}{2B(e-2)}. \quad (16c)$$

This is a linear programming problem. The optimal solution will be located at one of the following two vertices:

$$\lambda_2 = \beta + \frac{e}{e-1}(1 - \alpha - \beta), \quad \lambda_3 = 0, \quad (17a)$$

$$\lambda_2 = 1 - \alpha, \quad \lambda_3 = \frac{1 - \alpha - \beta}{2B(e-2)}, \quad (17b)$$

where λ_2 , and λ_3 are functions of α and β . However, determining the optimal solution between (17a) and (17b) requires further analysis. Therefore, we need to substitute (17a) and (17b) into (16a) separately to evaluate which yields the optimal result. By substituting (17a) into (16a), we obtain the following optimization problem:

$$\min_{\alpha, \beta} \left[-\frac{e}{e-1}(\mu_{B-} + q_{B+} B) + B \right] \alpha \\ + \left[-\frac{e}{e-1}(\mu_{B-} + q_{B+} B) + \mu_{B-} + 2q_{B+} B \right] \beta \\ + \frac{e}{e-1}(\mu_{B-} + q_{B+} B), \quad (18a)$$

$$\text{s.t. } \alpha + \beta \leq 1, 0 \leq \alpha, 0 \leq \beta. \quad (18b)$$

By substituting (17b) into (16a), we derive the following optimization problem:

$$\min_{\alpha, \beta} \left[-\frac{(2e-3)q_{B+} B}{2(e-2)} - \frac{\sigma_{B-}}{2B(e-2)} - \mu_{B-} + B \right] \alpha \\ + \left[-\frac{(2e-3)q_{B+} B}{2(e-2)} - \frac{\sigma_{B-}}{2B(e-2)} + 2q_{B+} B \right] \beta \\ + \frac{(2e-3)q_{B+} B}{2(e-2)} + \frac{\sigma_{B-}}{2B(e-2)} + \mu_{B-}, \quad (19a)$$

$$\text{s.t. } \alpha + \beta \leq 1, 0 \leq \alpha, 0 \leq \beta. \quad (19b)$$

Both (18a) and (19a) are linear programming problems with variables α and β . The primary distinction between (18a) and (19a) lies in the distinct forms of $p(t)$ used, as varying values of λ_3 lead to changes in $p(t)$ as expressed in (13a). The solutions and objective costs at each vertex for the linear programming problem in (18a) are summarized as follows:

$$\alpha = 1, \beta = 0, \mathcal{C}_{1,0} = B, \quad (20a)$$

$$\alpha = 0, \beta = 1, \mathcal{C}_{0,1} = \mu_{B-} + 2q_{B+} B, \quad (20b)$$

$$\alpha = 0, \beta = 0, \mathcal{C}_{0,0} = \frac{e(\mu_{B-} + q_{B+} B)}{e-1}. \quad (20c)$$

For the linear programming problem in (19a), the solutions and the associated objective costs at each vertex are summarized as follows:

$$\alpha = 1, \beta = 0, \mathcal{C}'_{1,0} = B, \quad (21a)$$

$$\alpha = 0, \beta = 1, \mathcal{C}'_{0,1} = \mu_{B-} + 2q_{B+} B, \quad (21b)$$

$$\alpha = 0, \beta = 0, \mathcal{C}'_{0,0} = \frac{(2e-3)q_{B+} B^2 + \sigma_{B-}}{2(e-2)} + \mu_{B-}. \quad (21c)$$

The resulting objective costs are identical in cases (20a) and (21a), and similarly, the objective costs are the same in cases (20b) and (21b). For these cases, we obtain $P(t)$ by substituting $\alpha = 1, \beta = 0$ and $\alpha = 0, \beta = 1$ into (4a), respectively. For cases (20c) and (21c), we substitute $\alpha = 0, \beta = 0$, (17a), (17b), (13a) and (13b) into (4a) to obtain $P(t)$, respectively. The strategy for the 2-slope ski rental problem determines $P(t)$ by evaluating the minimal objective

costs between $C_{1,0}$, $C_{0,1}$, $C_{0,0}$ and $C'_{0,0}$. Therefore, the strategy is summarized as follows:

$$P(t) = \begin{cases} \delta(t), & C_{1,0} < \min \{C_{0,1}, C_{0,0}, C'_{0,0}\}, \\ \delta(t - B), & C_{0,1} < \min \{C_{1,0}, C_{0,0}, C'_{0,0}\}, \\ \frac{1}{B(e-2)}(e^{t/B} - 1), & C'_{0,0} < \min \{C_{1,0}, C_{0,1}, C_{0,0}\}, \\ \frac{1}{B(e-1)}e^{t/B}, & \text{otherwise.} \end{cases} \quad (22a)$$

For the last two probability density functions, we integrate them to derive their respective cumulative distribution functions, as shown below:

$$\hat{P}(t) = \begin{cases} \frac{e^{t/B} - t/B - 1}{e - 2}, & 0 \leq t < B, \\ 1, & B \leq t. \end{cases} \quad (23a)$$

$$\hat{P}(t) = \begin{cases} \frac{e^{t/B} - 1}{e - 1}, & 0 \leq t < B, \\ 1, & B \leq t. \end{cases} \quad (23b)$$

The primary distinction between (23a) and (23b) lies in (23a) preference for retaining layers $\bar{i}+1$ to $|I|$ in memory. The remaining question is how to determine $\bar{t}$ based on the strategy. We set $\bar{t} = 0$ and $\bar{t} = B$, corresponding to the cases where $P(t) = \delta(t)$ and $P(t) = \delta(t - B)$, respectively. For the last two cases, we first sample from a $[0, 1]$ uniform distribution $\mathcal{U}[0, 1]$ to obtain the constant ξ and set $\bar{t}$ to the smallest t that satisfies $\xi \leq \hat{P}(t)$.

F. Layer-granularity Unloading Algorithm

Based on the above analysis, we summarize the LUA in Algorithm 1. LUA waits for an idle event (lines 3 - 4). Upon receiving the idle event, the process resumes, and all layers are currently retained in memory. LUA updates the statistics of the request arrival intervals (line 5). Subsequently, it identifies the smallest $\bar{i}$ such that $D_i \leq D_{QoS}$ meets the QoS constraint, ensuring that layers 1 to $\bar{i}$ are persistently retained in memory (line 6). The function COMPUTESTRATEGY is invoked to determine the unloading time for layers $\bar{i}+1$ to $|I|$. Layers $\bar{i}+1$ to $|I|$ are retained for a duration of $\bar{t}$ (lines 7 - 8). After that, these layers are unloaded following a timeout (lines 9 - 11). The layer unloading operation is terminated if a busy event is received during this period, indicating that the Inference Engine has received a request. Finally, LUA returns to line 3 to wait for the idle event (line 12). Additionally, in the dynamic workload scenarios, we introduce a sliding window approach to handle irregular arrivals. Specifically, we maintain a sliding window that records the inter-arrival intervals of the most recent requests. The window is updated upon each new request arrival, ensuring that statistics reflects the most recent workload dynamics.

COMPUTESTRATEGY incorporates the statistics q_{B+} , μ_{B-} , and σ_{B-} to determine the unloading time for layers $\bar{i}+1$ to $|I|$. It samples a constant ξ from a uniform distribution within the interval $[0, 1]$ (line 14). Subsequently, it calculates the corresponding objective costs based on the statistics of request arrival intervals (line 15). As analyzed in Section IV-E,

Algorithm 1 Layer-granularity Unloading Algorithm (LUA)

Input: $c_i, d_i, \eta, \mathcal{I}, D_{QoS}$

```

1: for  $i \in \mathcal{I} \cup \{0\}$  do
2:   Set  $C_i \leftarrow \sum_{i' \leq i} c_{i'}$  and  $D_i \leftarrow \sum_{i' > i} d_{i'}$ 
3: while No idle event received do
4:   Waiting for idle event
5: Update  $q_{B+}$ ,  $\mu_{B-}$ ,  $\sigma_{B-}$ , and  $t_{idle} \leftarrow CurrentTime$ 
6: Set  $\bar{i} \leftarrow \{\min_{i \in \mathcal{I}} \{i\} | D_i \leq D_{QoS}\}$  and  $B \leftarrow \eta D_{\bar{i}} / (C_{|\mathcal{I}|} - C_{\bar{i}})$ 
7: Get  $\bar{t} \leftarrow COMPUTESTRATEGY(B, q_{B+}, \mu_{B-}, \sigma_{B-})$ 
8: Set  $\{t_i \leftarrow +\infty | 1 \leq i \leq \bar{i}\}$  and  $\{t_i \leftarrow \bar{t} | \bar{i} + 1 \leq i \leq |\mathcal{I}|\}$ 
9: while No busy event received do
10:  if  $CurrentTime - t_{idle} \geq \bar{t}$  then
11:    Send the unload events to the Unloading Engine
    to unload layers  $(\bar{i} + 1) - (|\mathcal{I}|)$ 
12: Go to line 3
13: function COMPUTESTRATEGY( $B, q_{B+}, \mu_{B-}, \sigma_{B-}$ )
14:  Sample from uniform distribution  $\mathcal{U}[0, 1]$  to obtain  $\xi$ 
15:  Compute  $C_{1,0}$ ,  $C_{0,1}$ ,  $C_{0,0}$  and  $C'_{0,0}$  by (20a), (20b),
    (20c) and (21c), respectively
16:  if  $C_{1,0} < \min \{C_{0,1}, C_{0,0}, C'_{0,0}\}$  then
17:    Set  $\bar{t} \leftarrow 0$ 
18:  else if  $C_{0,1} < \min \{C_{1,0}, C_{0,0}, C'_{0,0}\}$  then
19:    Set  $\bar{t} \leftarrow B$ 
20:  else if  $C'_{0,0} < \min \{C_{1,0}, C_{0,1}, C_{0,0}\}$  then
21:    Compute  $\hat{P}(t)$  by (23a)
22:    Set  $\bar{t} \leftarrow \{\min \{t\} | \xi \leq \hat{P}(t)\}$ 
23:  else
24:    Compute  $\hat{P}(t)$  by (23b)
25:    Set  $\bar{t} \leftarrow \{\min \{t\} | \xi \leq \hat{P}(t)\}$ 
return  $\bar{t}$ 

```

COMPUTESTRATEGY derives $\bar{t}$ based on $P(t)$ by evaluating the minimal objective costs among $C_{1,0}$, $C_{0,1}$, $C_{0,0}$, and $C'_{0,0}$ (lines 16 - 25).

G. Time Complexity Analysis

We now analyze the time complexity of LUA. Line 6 selects the index $\bar{i}$ that satisfies the QoS constraint, which requires $O(|I|)$ time. Subsequently, updating unloading timers also requires $O(|I|)$ time. The COMPUTESTRATEGY function (lines 13 - 25) consists of the following operations: sampling from a uniform distribution ($O(1)$), computing the corresponding objective costs ($O(1)$), and setting $\bar{t}$ ($O(1)$). Therefore, the overall worst-case time complexity is $O(|I|)$. The low overhead of LUA is further validated through runtime measurements on an NVIDIA Jetson AGX Orin device, where its decision-making process introduces an overhead of only about 1 ms.

V. PERFORMANCE EVALUATION

A. Evaluation Setup

Testbed Environment. We assess LUA's performance and scalability across heterogeneous edge environments by deploy-

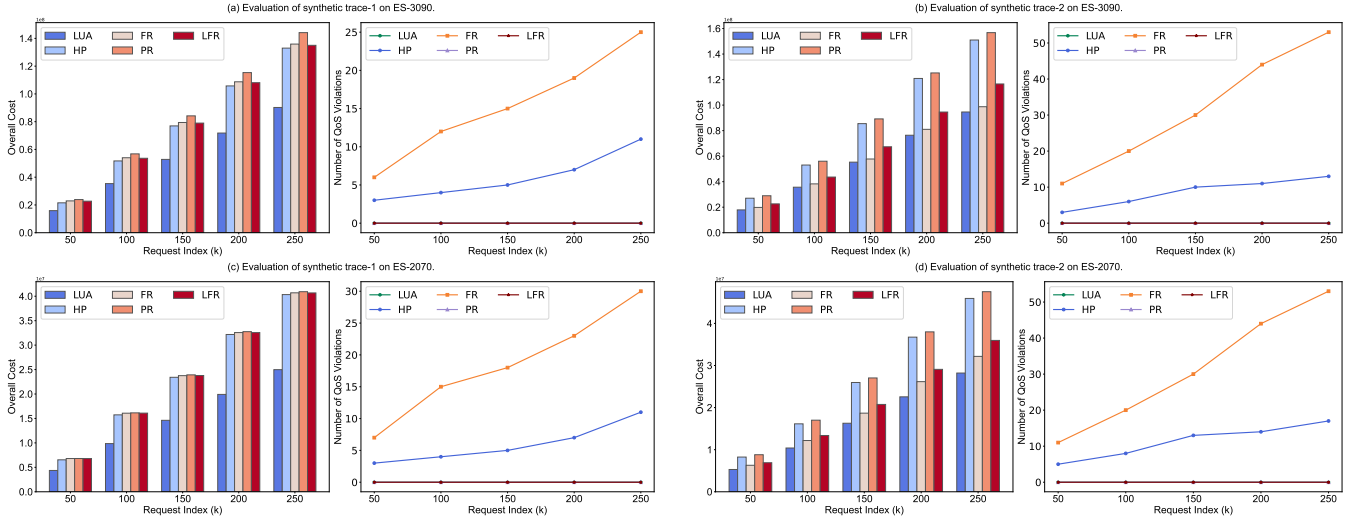


Fig. 4. Performance evaluation results for synthetic workloads.

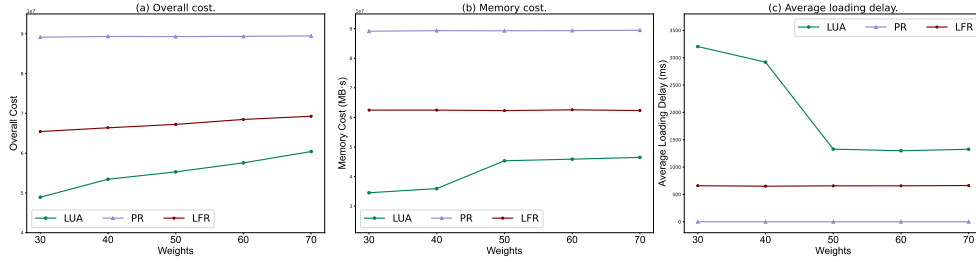


Fig. 5. Performance evaluation results for different weights.

ing it on multiple edge servers with the following specifications:

- Edge Server 3090 (ES-3090): This edge server is equipped with an Intel(R) Xeon(R) Silver 4210 CPU @ 2.20GHz, a NVIDIA GeForce RTX 3090 GPU with 24 GB GDDR6X memory, and PCIe 3.0.
- Edge Server 2070 (ES-2070): This edge server is equipped with an Intel(R) Core(TM) i9-10900K CPU @ 3.70GHz, a NVIDIA GeForce RTX 2070 SUPER GPU with 8 GB GDDR6 memory, and PCIe 3.0.
- Edge Server Orin (ES-Orin): The NVIDIA Jetson AGX Orin edge server is equipped with 64 GB LPDDR5 memory, and PCIe 4.0.

Workload. Our evaluation employs both synthetic workload traces and real-world workload traces [37]. The synthetic trace-1 and trace-2 are generated according to the Normal [38] and Pareto [39] distributions, respectively. For the Normal distribution, the mean and standard deviation are set to 35 and 25, respectively. The Pareto distribution's minimum possible value, shape, and scale parameters are set to 1, 0.5, and 2, respectively. Any non-positive values and large outliers in the generated requests' time interval are removed. The dynamic real-world workload traces consist of timestamped user requests to GPT-4 collected over a two-month period from a university campus. We segment the real-world workload traces into four different traces: real-world trace-1 to trace-4.

Model. To evaluate the scalability of LUA across model

TABLE III
LAYER'S LOADING DELAY AND MEMORY FOOTPRINT

Layers	Loading delay	Memory footprint
Embedding layer	380 ms	1188 MB
Transformer layer	125 ms	386 MB
Fully connected layer	380 ms	1188 MB
Total	4760 ms	14728 MB

sizes, we employ the Qwen-7B-Chat model (running on ES-3090 and ES-Orin) and the Qwen-1.8B model [40] (running on ES-2070) as representative examples of the LLM. The Qwen-7B-Chat model comprises an embedding layer, 32 transformer layers, a normalization layer, and a fully connected layer. Detailed measurements of each layer's loading delay and memory footprint on the ES-3090 for the model loaded with float16 precision are presented in TABLE III. The values presented are averages of multiple measurements to account for potential millisecond-level fluctuations. The normalization layer's loading delay and memory footprint are negligible. The QoS constraints are set to 2500–4500 ms for the Qwen-7B-Chat model, while a tighter QoS constraint of 700 ms is applied to the Qwen-1.8B model.

B. Baselines

- Histogram Policy (HP) [13]. HP maintains a histogram of historical inter-arrival times of user requests. The model retention time is set to the 95th percentile of the observed

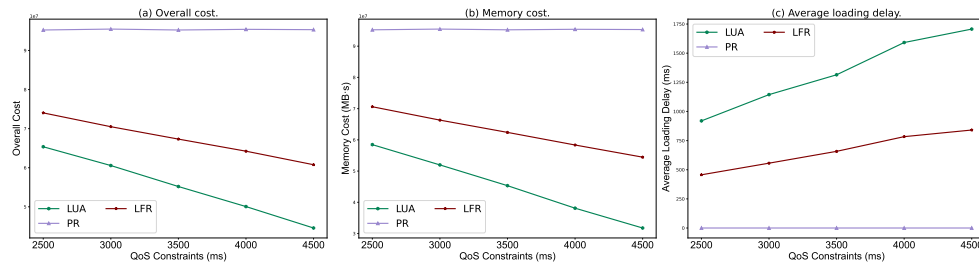


Fig. 6. Performance evaluation results for different QoS constraints.

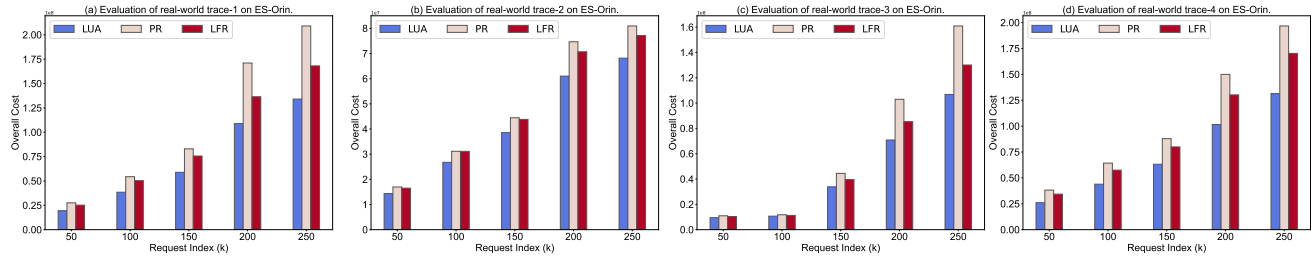


Fig. 7. Performance evaluation results for heterogeneous edge devices with real-world dynamic workloads.

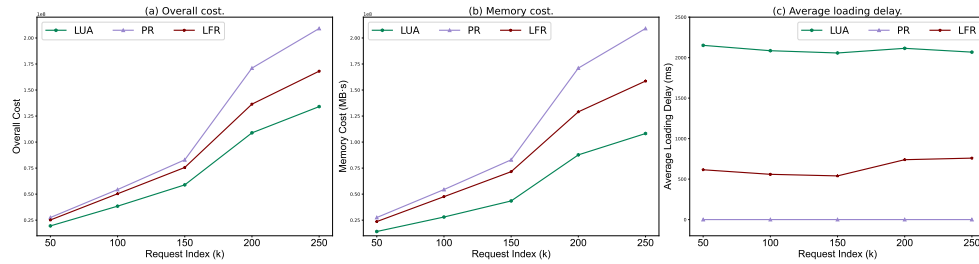


Fig. 8. Evaluation of real-world trace-1 on ES-Orin in terms of average loading delay and memory cost decomposition.

request intervals. If no new request arrives within this threshold, the model is unloaded.

- Fixed Retention (FR) [14]. It retains the model for a fixed period. In this paper, FR is implemented by assigning a tag of 60 to the model. During the model's idle period, this tag is decremented by 1 every second until it reaches 0, at which point the model is unloaded.
- Persist Retention (PR). This method, commonly employed in model service systems [8], persistently retains the model in memory without unloading it.
- Layer-granularity Fixed Retention (LFR). LFR extends FR to the layer granularity. Specifically, it first identifies the minimum set of layers (i.e., consistent with the LUA) that must always be retained to satisfy the QoS constraint on loading delay. These layers are persistently kept in memory, while the remaining layers are unloaded using a fixed retention timer, similar to FR.

C. Evaluation Results

Overall Performance. The trade off of memory cost and loading delay is reflected in the reduction of the objective function's overall cost. To this end, we conduct experiments using different synthetic workload traces to evaluate LUA's overall performance, as illustrated in Fig. 4. Here, we use the request index to denote the position of a request in the

arrival sequence. For example, $k = 50$ represents the system performance observed when the 50th request completes. The proposed LUA consistently demonstrates better performance across both synthetic traces, strictly adhering to the QoS constraint. Compared to the widely used HP and PR methods, LUA reduces the overall cost by up to 37.3% and 39.6%, respectively. HP and FR experience significant loading delays due to some requests arriving after these methods have already unloaded the model, resulting in QoS violations. While the PR method does not violate the QoS constraint, it incurs a high memory cost, leading to the poorest overall performance. The benefits of layer granularity in retention and unloading are evident when comparing the overall cost of LFR with PR, as LFR maintains relatively low overall costs without violating the QoS constraint. Additionally, a comparison between LFR and LUA shows up to a 33.1% reduction in overall costs, further demonstrating the advantage of the proposed unloading strategy. In summary, by employing layer-granularity unloading strategy, LUA jointly minimizes loading delay and memory costs while meeting the QoS constraint, thus demonstrating superior overall performance.

Our design primarily targets dense transformer models, where each inference requires all layers to be resident in memory. The proposed method addresses a general parameter residency management problem: given that the arrival times of

future requests are unknown, deciding whether model's parameters that needed by future requests should remain in memory or be unloaded to save memory cost. In Mixture-of-Experts (MoE) architectures (e.g., DeepSeek [41]), only a subset of experts is activated. In this case, a group of experts within the layer can be treated as a parameter unit, and the same unloading decision can be applied. Therefore, the proposed approach remains applicable at a coarser granularity. However, note that some MoE serving systems may schedule active and inactive experts independently based on routing decisions [42], in which the inactive experts do not need to be loaded into memory. Integrating such expert-aware scheduling with our method introduces a finer-grained scheduling problem, which we leave as future work.

Effect of Different Weights. We vary the weight η from 30 to 70 to evaluate its impact on system performance, as shown in Fig. 5. Due to significant QoS violations, HP and FR are excluded from this and subsequent evaluations. From Fig. 5(a), it is evident that LUA achieves better overall cost while satisfying the QoS constraint. Compared to the widely used PR method, LUA reduces overall costs by up to 45.1%. As illustrated in Fig. 5(b) and Fig. 5(c), regardless of the variation in η , PR and LFR perform model retention and unloading according to a fixed strategy, thus exhibiting constant memory cost and average loading delay. A larger η increases the cost incurred by loading delays, making retaining the layers more profitable. As η increases, LUA tends to retain the layers for longer durations. This explains why LUA reduces the average loading delay by consuming more memory cost. However, an excessively large η leads to prohibitively high memory costs, which is detrimental for edge servers with relatively limited resources. Therefore, careful determination of η is necessary. Although our current implementation sets η empirically, future work may explore automated strategies to dynamically adjust η based on observed workload characteristics and system feedback. In summary, the results demonstrate that LUA can effectively adjust the layer unloading strategy in response to changes in η to trade off the loading delay and memory cost. For instance, when edge server resources are scarce, reducing η allows LUA to decrease layer retention to save memory resources and vice versa.

Effect of Different QoS Constraints. We set the QoS constraints between 2500 ms and 4500 ms to evaluate their impact on performance, as shown in Fig. 6. As depicted in Fig. 6(a), LUA achieves an overall cost reduction of up to 26.6% and 53.3%, respectively, compared to LFR and PR. Fig. 6(b) illustrates that to meet the QoS constraint, PR had to retain the entire model for extended periods due to their lack of awareness of the model's layer structure. This led to substantial memory costs and consequently poor overall performance. Conversely, LUA can make informed decisions on when to unload layers, jointly minimizing memory costs and loading delays, thus achieving better overall performance. In Fig. 6(b), as the QoS constraint becomes more relaxed, both LUA and LFR retain fewer layers in memory. This explains why, as the QoS constraint becomes more relaxed, the memory costs of both LUA and LFR are reduced. Fig. 6(c) shows that a more relaxed QoS constraint leads to higher average loading

delays as LUA and LFR retain fewer layers in memory; thus, more layers are required for reloading.

Performance Evaluation on Heterogeneous Edge Device with Real-world Dynamic Workloads. To further evaluate the scalability of the proposed approach, we use real-world workload traces on a heterogeneous edge platform, NVIDIA Jetson AGX Orin. Unlike discrete platforms, ES-Orin adopts a unified memory architecture, where the CPU and GPU share system memory. As shown in Fig. 7 and Fig. 8, we replay the real-world traces and plot the online performance. PR and LFR keep layers resident for long periods, which reduces average loading delay but significantly increases memory cost during sparse request intervals. By using the sliding window to capture recent arrival dynamics and making decision based on statistics, LUA exhibits different behavior. During bursty arrivals, it avoids unnecessary unloading and reloading; during prolonged idle intervals, it unloads layers accordingly to reduce memory cost. The sliding window effectively addresses dynamic workload issues by continuously monitoring the statistics of recent requests to dynamically adjust the unloading strategy of layers. As a result, LUA achieves a lower overall cost than PR and LFR across all traces. These results demonstrate that the proposed LUA is scalable to both heterogeneous edge resource configurations and dynamic real-world workloads.

VI. CONCLUSION

This paper proposed a layer-granularity unloading approach for LLMs in edge computing. We formulated an online joint optimization problem to decide which layers need to be unloaded and when to unload the layers, in which the overall cost is defined as the weighted sum of loading delay and memory cost. Then, we introduced LUA to solve it, which is based on the 2-slope ski rental model and incorporates statistics. By carefully determining the unloading strategy, LUA efficiently trades off loading delays and memory costs while satisfying the QoS constraint. Moreover, a layer-granularity unloading system was implemented on a test bed with different NVIDIA GPUs. Extensive experiments demonstrated LUA's effectiveness, showing a reduction in the overall cost by up to 39.6%. Additionally, LUA excelled in heterogeneous environments, highlighting its scalability. Future work will scale our method to support edge-cloud collaborative LLM services and MoE models, aiming for improved cost-effective deployment.

REFERENCES

- [1] B. Min, H. Ross, E. Sulem *et al.*, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.
- [2] J. Yang, H. Jin, R. Tang *et al.*, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 6, pp. 1–32, 2024.
- [3] M. Xu, H. Du, D. Niyato *et al.*, "Unleashing the power of edge-cloud generative ai in mobile networks: A survey of aigc services," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1127–1170, 2024.
- [4] Z. Tang, F. Mou, J. Lou, W. Jia, Y. Wu, and W. Zhao, "Joint resource overbooking and container scheduling in edge computing," *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 10903–10917, 2024.

- [5] H. Cui, Z. Tang, J. Lou, W. Jia, and W. Zhao, "Latency-aware container scheduling in edge cluster upgrades: A deep reinforcement learning approach," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2530–2543, 2024.
- [6] A. Paszke, S. Gross, F. Massa *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS 19)*. Curran Associates, Inc., 2019, pp. 8024–8035.
- [7] A. Vaswani, N. Shazeer, N. Parmar *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS 17)*. Curran Associates, Inc., 2017, pp. 6000–6010.
- [8] D. Crankshaw, X. Wang, G. Zhou *et al.*, "Clipper: A {Low-Latency} online prediction serving system," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX, 2017, pp. 613–627.
- [9] G. K. Walia, M. Kumar, and S. S. Gill, "Ai-empowered fog/edge resource management for iot applications: A comprehensive review, research challenges and future perspectives," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 1, pp. 619–669, 2023.
- [10] Y. Qiao, S. Anzai, S. Yu, H. Ma, Y. Wang, M. Kim, and H. Xu, "Conserve: Harvesting gpus for low-latency and high-throughput large language model serving," *arXiv preprint arXiv:2410.01228*, 2024.
- [11] S. Athlur, N. Saran, M. Sivathanu, R. Ramjee, and N. Kwatra, "Varuna: scalable, low-cost training of massive deep learning models," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys 22)*, 2022, pp. 472–487.
- [12] M. Shahrad, R. Fonseca, I. Goiri *et al.*, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (ATC 20)*. USENIX, 2020, pp. 205–218.
- [13] Z. Li, L. Guo, Q. Chen *et al.*, "Help rather than recycle: Alleviating cold startup in serverless computing through {Inter-Function} container sharing," in *2022 USENIX Annual Technical Conference (ATC 22)*. USENIX, 2022, pp. 69–84.
- [14] Amazon, "Aws lambda," 2014. [Online]. Available: <https://aws.amazon.com/lambda/>
- [15] Z. Yu, Z. Wang, Y. Li, R. Gao, X. Zhou, S. R. Bommur, Y. Zhao, and Y. Lin, "Edge-llm: Enabling efficient large language model adaptation on edge devices via unified compression and adaptive layer voting," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC 24)*. ACM, 2024, pp. 1–6.
- [16] R.-G. Dumitru, V. Yadav, R. Maheshwary *et al.*, "Variable layer-wise quantization: A simple and effective approach to quantize llms," *arXiv preprint arXiv:2406.17415*, 2024.
- [17] G. Li, "Airlm: scaling large language models on low-end commodity computers," 2023. [Online]. Available: <https://github.com/lyogavin/airllm/>
- [18] S. Guggler, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate," 2022. [Online]. Available: <https://github.com/huggingface/accelerate>
- [19] S. Shukla and A. A. Abouzeid, "Proactive retention aware caching," in *IEEE INFOCOM 2017-IEEE Conference on Computer Communications (INFOCOM 17)*. IEEE, 2017, pp. 1–9.
- [20] L. Pan, L. Wang, S. Chen, and F. Liu, "Retention-aware container caching for serverless edge computing," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications (INFOCOM 22)*. IEEE, 2022, pp. 1069–1078.
- [21] E. Hosseini, M. Nickray, and S. Ghanbari, "Optimized task scheduling for cost-latency trade-off in mobile fog computing using fuzzy analytical hierarchy process," *Computer Networks*, vol. 206, p. 108752, 2022.
- [22] Y. Guan, X. Zhang, and Z. Guo, "Prefcache: Edge cache admission with user preference learning for video content distribution," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 4, pp. 1618–1631, 2020.
- [23] Z. Lotker, B. Patt-Shamir, and D. Rawitz, "Rent, lease, or buy: Randomized algorithms for multislope ski rental," *SIAM Journal on Discrete Mathematics*, vol. 26, no. 2, pp. 718–736, 2012.
- [24] M. Xu, D. Niyato, H. Zhang *et al.*, "Cached model-as-a-resource: Provisioning large language model agents for edge intelligence in space-air-ground integrated networks," *arXiv preprint arXiv:2403.05826*, 2024.
- [25] G. Qu, Z. Lin, F. Liu, X. Chen, and K. Huang, "Trimcaching: Parameter-sharing ai model caching in wireless edge networks," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS 24)*. IEEE, 2024, pp. 36–46.
- [26] Y. Sheng, S. Cao, D. Li *et al.*, "S-lora: Serving thousands of concurrent lora adapters," *arXiv preprint arXiv:2311.03285*, 2023.
- [27] A. Jaiswal, K. S. Shahana, S. Ravichandran, K. Adarsh, H. B. Bhat, B. K. Joardar, and S. K. Mandal, "Halo: Communication-aware heterogeneous 2.5 d system for energy-efficient llm execution at edge," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 14, no. 3, pp. 425–439, 2024.
- [28] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "Edgeshard: Efficient llm inference via collaborative edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 119–13 131, 2025.
- [29] Z. Yao, Z. Tang, J. Lou *et al.*, "Velo: A vector database-assisted cloud-edge collaborative llm qos optimization framework," in *Proceedings of the 2024 IEEE International Conference on Web Services (ICWS 24)*. IEEE, 2024, pp. 1–12.
- [30] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, and M. Yang, "T-mac: Cpu renaissance via table lookup for low-bit llm deployment on edge," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys 25)*. ACM, 2025, pp. 278–292.
- [31] Z. Yang, Y. Yang, C. Zhao *et al.*, "Perllm: Personalized inference scheduling with edge-cloud collaboration for diverse llm services," *arXiv preprint arXiv:2405.14636*, 2024.
- [32] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Transactions on Mobile Computing*, pp. 1–12, 2024.
- [33] H. Face, "Safetensors," 2022. [Online]. Available: <https://github.com/huggingface/safetensors>
- [34] A. Khanafer, M. Kodialam, and K. P. Puttaswamy, "The constrained ski-rental problem and its application to online cloud cost optimization," in *2013 Proceedings IEEE INFOCOM (INFOCOM 13)*. IEEE, 2013, pp. 1492–1500.
- [35] C. Dong, H. Zeng, and M. Chen, "A cost efficient online algorithm for automotive idling reduction," in *Proceedings of the 51st Annual Design Automation Conference (DAC 14)*. ACM, 2014, pp. 1–6.
- [36] A. Khanafer, M. Kodialam, and K. P. Puttaswamy, "To rent or to buy in the presence of statistical information: The constrained ski-rental problem," *IEEE/ACM Transactions on Networking*, vol. 23, no. 4, pp. 1067–1077, 2014.
- [37] Y. Wang, Y. Chen, Z. Li, X. Kang, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou *et al.*, "Burstgpt: A real-world workload dataset to optimize llm serving systems," *arXiv preprint arXiv:2401.17644*, 2024.
- [38] J.-W. Sung and S.-J. Han, "Data bundling for energy efficient communication of wearable devices," *Computer Networks*, vol. 121, pp. 76–88, 2017.
- [39] J.-H. Kim, G. S. Choi, and C. R. Das, "Coscheduled distributed-web servers on system area network," *Journal of Parallel and Distributed Computing*, vol. 68, no. 8, pp. 1033–1043, 2008.
- [40] A. Cloud, "Qwen-7b-chat," 2023. [Online]. Available: <https://huggingface.co/Qwen>
- [41] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [42] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.



Zhenzheng Li received the B.S. degree from School of Information Technology, Beijing Institute of Technology, Zhuhai, China, in 2019, and the M.S. degree from School of Information and Control Engineering, China University of Mining and Technology, China, in 2022. He is currently pursuing the Ph.D. degree in School of Artificial Intelligence, Beijing Normal University, China. His current research interests include edge computing and resource scheduling.



Zhiqing Tang received the B.S. degree from the School of Communication and Information Engineering, University of Electronic Science and Technology of China, China, in 2015 and the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in 2022. He is currently an Assistant Professor at the Advanced Institute of Natural Sciences, Beijing Normal University, China. His research interests include edge computing, resource scheduling, and reinforcement learning.



Wei Zhao (Fellow, IEEE) completed his undergraduate studies in physics at Shaanxi Normal University, China, in 1977, and received his MSc and PhD degrees in Computer and Information Sciences at the University of Massachusetts at Amherst in 1983 and 1986, respectively. Professor Zhao has served important leadership roles in academic including the Chief Research Officer at the American University of Sharjah, the Chair of Academic Council at CAS Shenzhen Institute of Advanced Technology, the eighth Rector of the University of Macau, the Dean of Science at Rensselaer Polytechnic Institute, the Director for the Division of Computer and Network Systems in the U.S. National Science Foundation, and the Senior Associate Vice President for Research at Texas A&M University. Professor Zhao has made significant contributions to cyber-physical systems, distributed computing, real-time systems, and computer networks. He led the effort to define the research agenda of and to create the very first funding program for cyber-physical systems in 2006. His research results have been adopted in the standard of Survivable Adaptable Fiber Optic Embedded Network. Professor Zhao was awarded the Lifelong Achievement Award by the Chinese Association of Science and Technology in 2005.



Jianxiong Guo received his Ph.D. degree from the Department of Computer Science, University of Texas at Dallas, Richardson, TX, USA, in 2021, and his B.E. degree from the School of Chemistry and Chemical Engineering, South China University of Technology, Guangzhou, China, in 2015. He is currently an Associate Professor with the Advanced Institute of Natural Sciences, Beijing Normal University, and also with the Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, Zhuhai, China. He is

a member of IEEE/ACM/CCF. His research interests include social networks, wireless sensor networks, combinatorial optimization, and machine learning.



Weijia Jia (Fellow, IEEE) is currently the Director of Institute of Artificial Intelligence and Future Networking, and the Director of Super Intelligent Computer Center, Beijing Normal University at Zhuhai; also a Chair Professor at Beijing Normal-Hong Kong Baptist University, Zhuhai, Guangdong, China. He has served as the VP for Research at UIC in 6/2020-7/2024. Prior joining BNU, he served as the Deputy Director of State Key Laboratory of Internet of Things for Smart City at the University of Macau and Zhiyuan Chair Professor at Shanghai Jiaotong

University, PR China. He received BSc/MSc from Center South University, China in 82/84 and PhD from Polytechnic Faculty of Mons, Belgium in 93, respectively; all in computer science. For 93-95, he joined German National Research Center for Information Science (GMD) in Bonn (St. Augustine) as a research fellow. From 95-13, he worked in City University of Hong Kong as a professor. His contributions have been recognized for the research of edge AI, optimal network routing and deployment; vertex cover; anycast and multicast protocols; sensors networking; knowledge relation extractions; NLP and intelligent edge computing. He has over 700 publications in the prestige international journals/conferences and research books and book chapters. He has received the best product awards from the International Science & Tech. Expo (Shenzhen) in 2011/2012 and the 1st Prize of Scientific Research Awards from the Ministry of Education of China in 2017 (list 2), and top 2% World Scientists in Stanford-list (2020-2024) and many provincial science and tech awards. He has served as area editor for various prestige international journals, chair and PC member/keynote speaker for many top international conferences. He is the Fellow of IEEE and the Distinguished Member of CCF.